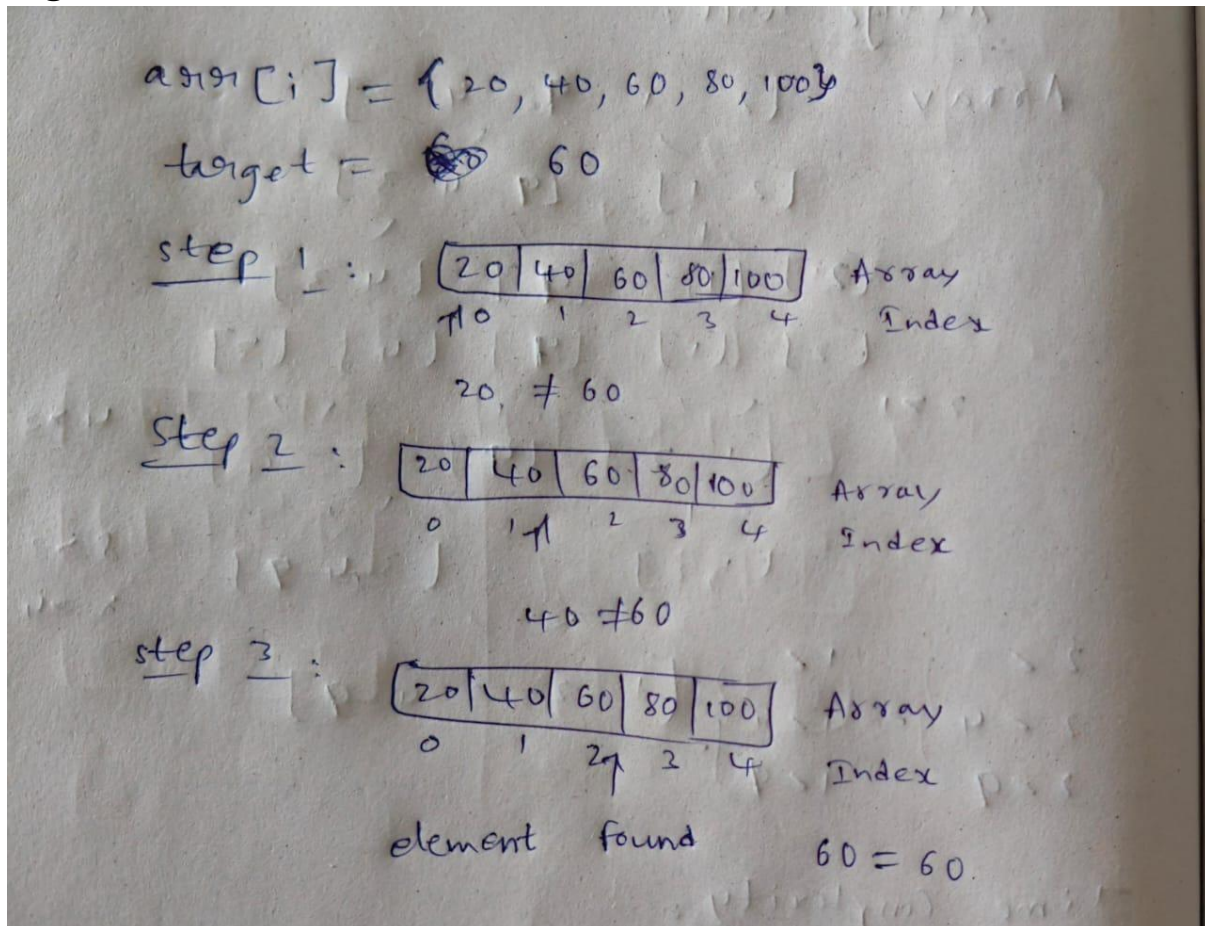


Practical - 2

Aim: To Implementation and Time analysis of linear and binary search Algorithms.

Algorithm for Linear Search:



array[i] = {20, 40, 60, 80, 100}

target = 60

step 1 :

20	40	60	80	100
0	1	2	3	4

Array Index

20 ≠ 60

step 2 :

20	40	60	80	100
0	1	2	3	4

Array Index

40 ≠ 60

step 3 :

20	40	60	80	100
0	1	2	3	4

Array Index

element found 60 = 60

Program of Linear Search:

```
#include <iostream>
```

```
using namespace std;
```

```
class LinearSearchApp
```

```
{
```

```
public:
```

```
    int arr[100], n;
```

```
    void arrayInput()
```

```
{
```

Name: B. Manoj Karthik

Enrollment Number: 92460118094



```
cout << "Enter number of elements: ";  
cin >> n;  
  
cout << "Enter the elements: ";  
for (int i = 0; i < n; i++)  
{  
    cin >> arr[i];  
}  
}  
  
void printArray()  
{  
    for (int i = 0; i < n; i++)  
    {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
}  
  
// Linear Search Function  
int search(int target)  
{  
    for (int i = 0; i < n; i++)  
    {  
        if (arr[i] == target)  
        {  
            return i;  
        }  
    }  
    return -1;  
}
```

```
    }  
};  
  
int main()  
{  
    LinearSearchApp ls;  
    int target, result;  
  
    ls.arrayInput();  
  
    cout << "\nArray elements: ";  
    ls.printArray();  
  
    // Linear Search Section  
    cout << "\nEnter element to search: ";  
    cin >> target;  
  
    result = ls.search(target);  
    if (result != -1)  
    {  
        cout << "Element " << target << " found at index: " << result << endl;  
    }  
    else  
    {  
        cout << "Element " << target << " not found in the array." << endl;  
    }  
  
    return 0;  
}
```

Output:

```
Enter number of elements: 5
Enter the elements: 20 40 60 80 100

Array elements: 20 40 60 80 100

Enter element to search: 60
Element 60 found at index: 2
```

Time complexity:

```
int search (int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) {
            return i;
        }
    }
    return -1;
}
```

Time complexity $T(n) = n + 1 + 1$
 $= n + 2$
 $\Rightarrow T(n) = O(n)$



i) Best case: - $O(1)$

ex: arr = [10, 20, 30, 40, 50]

key = 10, there is only one time compare

$$T(n) = 1$$

Time complexity = $O(1)$

ii) Average case: - $O(n)$

element found at in middle

ex: arr = {10, 20, 30, 40, 50}
0 1 2 3 4

key = 30 $\rightarrow \frac{n}{2}$ compare

$$\frac{5}{2} = 2 \text{ . arr[2], arr[2] = 30}$$

$$T(n) = \frac{n}{2}$$

$$T(n) = O(n) //$$

iii) Worst case - $O(n)$

element found at last

ex: arr {10, 20, 30, 40, 50}
0 1 2 3 4

key = 50, we check all element upto n

$$T(n) = n$$

$$T(n) = O(n)$$

Time complexity for linear search is $O(n) //$



Binary Search Algorithm:

Binary Search :-

Array [20, 40, 60, 80, 100]

target = 80,

20	40	60	80	100
0	1	2	3	4

↓

60	80	100
----	----	-----

↓

80	100
----	-----

↓

found

80

low = 0
high = 4
$$\text{mid} = \frac{0 + (4 - 0)}{2}$$
$$= 2$$

checking
 $\text{arr}[3] = \text{target}$

Program of Binary Search Algorithm:

```
#include <iostream>
using namespace std;
```

```
class BinarySearchApp
{
public:
    int arr[100], n;

    void arrayInput()
    {
```



```
cout << "Enter number of elements: ";  
  
cin >> n;  
  
cout << "Enter the elements in sorted order: ";  
for (int i = 0; i < n; i++)  
{  
    cin >> arr[i];  
}  
}  
  
void printArray()  
{  
    for (int i = 0; i < n; i++)  
    {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
}  
  
// Binary Search Function  
int search(int target)  
{  
    int low = 0;  
    int high = n - 1;  
  
    while (low <= high)  
    {  
        int mid = low + (high - low) / 2;  
  
        if (arr[mid] == target)
```



```
{  
    return mid;  
}  
else if (arr[mid] < target)  
{  
    low = mid + 1;  
}  
else  
{  
    high = mid - 1;  
}  
}  
return -1;  
}  
};
```

```
int main()  
{  
    BinarySearchApp bs;  
    int target, result;  
  
    bs.arrayInput();  
  
    cout << "\nArray elements: ";  
    bs.printArray();  
  
    // Binary Search Section  
    cout << "\nEnter element to search: ";  
    cin >> target;
```

```
result = bs.search(target);
if (result != -1)
{
    cout << "Element " << target << " found at index: " << result << endl;
}
else
{
    cout << "Element " << target << " not found in the array." << endl;
}

return 0;
}
```

Output:

```
Enter number of elements: 5
Enter the elements in sorted order: 20 40 60 80 100

Array elements: 20 40 60 80 100

Enter element to search: 80
Element 80 found at index: 3
```



Time Complexity:

Time complexity:

$$T(n) = 1 + 1 + \log n + 1 + 1 + 1$$

$$T(n) = \log n + 5$$

$$\boxed{T(n) = O(\log n)}$$

i) Best case - $O(1)$
 $\text{mid} = (\text{low} + \text{high})/2 \rightarrow 1$
 $\text{if } (a[\text{mid}] == \text{key}) \rightarrow 1$
 $T(n) = 1 + 1 = 2$
 $T(n) = O(1)$

ii) Average case - $O(\log n)$
 $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \dots$
 $\text{no. of divisions} = \log_2 n$
 $T(n) = \log_2 n$
 $T(n) = O(\log n)$

iii) Worst case - $O(\log n)$
 $n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \frac{n}{8} \dots 1$
 $T(n) = \log_2 n$
 $T(n) = O(\log n)$

Result: Successfully Implemented and done the Time analysis of Linear and Binary Search algorithm.